

Introduction to Python

Prepared By

Dr. Islam Zakaria

Lecturer at the Information Technology Department, FCAIH, Capital University

Definition

❑ What is Python?

- **High-level:** It abstracts away low-level hardware details like manual memory management, allowing you to focus on logic.
- **Interpreted:** Code is executed line-by-line by the interpreter rather than being compiled into machine code beforehand.
- **Multi-paradigm:** It supports multiple coding styles: Object-Oriented (classes), Procedural (functions), and Functional.
- **Dynamically Typed:** You do not need to declare variable types (e.g., `int x`); the type is determined at runtime.
- **Garbage-collected:** Python automatically handles memory allocation and de-allocation.

Core Philosophy – Readability

❑ The “art of Python”

- The language is designed with the philosophy that code is read much more often than it is written.

❑ Significant Indentation:

- Python uses whitespace (indentation) to delimit code blocks.
- This stands in contrast to languages like C, C++, or Java that use curly braces {}.

❑ Impact on Code:

- **Clean Visual Structure:** Enforces a uniform layout for all developers.
- **Reduced Noise:** Removes unnecessary punctuation (semicolons, braces), making the code less cluttered.
- **Maintainability:** Easier to read and debug by others.

Core Philosophy – Simplicity

❑ Executable Pseudocode

- Python syntax mimics natural language and mathematical notation.
- The focus remains on the solution (algorithm), not the syntax.

❑ Efficiency of Expression:

- Developers can express complex concepts in fewer lines of code compared to lower-level languages.
- Reduces cognitive load and development time.

❑ Example (Swapping Variables):

- **Python:** `a, b = b, a`
- **C/Java:** Requires a temporary variable (`temp = a; a = b; b = temp;`).

Core Philosophy – Extensibility

❑ Ecosystem & Libraries:

- Python supports a vast system of modules and packages, allowing it to integrate with other languages and tools.“

❑ Batteries Included":

- The standard library covers many common tasks (regex, math, HTTP) without needing external installations.

❑ Domain Dominance:

- **Data Science:** NumPy (numerical computing), Pandas (tabular data manipulation).
- **Web Development:** Django, Flask.
- **Artificial Intelligence:** PyTorch, TensorFlow.

Syntax Basics (Variables)

❑ Concept: The Variable

- **Definition:** A variable in Python is a symbolic name that is a reference or pointer to an object. Unlike C++ or Java, where a variable is a specific storage location in memory with a fixed type, Python variables are tags mapped to objects.
- **Initialization:** Variables are created the moment you assign a value to them using the = operator.
- **Dynamic Typing:** You do not declare the type (e.g., `int x`). Python infers the type at runtime based on the value assigned.

Naming Conventions

- ❑ Variable names must use snake_case (e.g., app_name, total_cost).
- ❑ They must start with a letter or an underscore _, **never a number**.
- ❑ They are case-sensitive (**Price and price are different variables**).

```
# Initializing variables
app_name = "Facebook"      # String
price = 0.0                # Float
user_ratings = 2974676     # Integer

# Updating variables (Re-assignment)
user_ratings = user_ratings + 1
```

Syntax Basics (Arithmetic & Comments)

❑ The Interpreter's Logic:

- Python follows standard mathematical order of operations (**PEMDAS**: Parentheses, Exponents, Multiplication/Division, Addition/Subtraction).

❑ Arithmetic Operators:

- **+** (Addition), **-** (Subtraction), ***** (Multiplication), **/** (Division - always returns a float).
- ****** (Exponentiation): Raising a number to a power.
- **//** (Floor Division): Returns the integer part of the division (rounding down).

❑ Code Documentation (Comments):

- **Symbol:** **#**
- **Purpose:** Text following **#** is ignored by the interpreter. It is crucial for explaining complex logic or "legacy code" to other developers.

Example (Calculation):

```
# Calculate revenue: Price * Downloads
price = 1.99
downloads = 500
revenue = price * downloads # Output: 995.0

# Complex Operation using Parentheses
# ((Base + Bonus) squared) divided by 10
score = ((4 + 8) ** 2) / 10 # Output: 14.4
```

Data Types (Integers & Floats)

❑ Numeric Primitives:

- **Integers (int):** Whole numbers without a fractional component. In Python 3, integers have arbitrary precision (they can be as large as memory allows).
- **Floating Point Numbers (float):** Numbers with a decimal point. They are implemented using double-precision (64-bit) binary format (IEEE 754).
- **Academic Note:** Floats can sometimes be imprecise due to binary representation (e.g., $0.1 + 0.2$ might equal 0.30000000000000004).

❑ Type Inspection:

- The `type()` function is used to introspect the data type of any variable at runtime.

Data Types (Integers & Floats)

❑ Example:

```
count = 50          # int
rating = 4.5         # float

print(type(count))  # Output: <class 'int'>
print(type(rating)) # Output: <class 'float'>

# Division always results in a float in Python 3
result = 20 / 4      # Output: 5.0 (float), not 5 (int)
```

Data Types (Strings & Casting)

❑ Strings (str):

- A sequence of Unicode characters enclosed in single ' or double " quotes.
- **Immutability:** Once created, a string object cannot be changed. Operations like concatenation create a new string.

❑ Type Casting (Explicit Conversion):

- **Definition:** Forcing a variable of one type to behave like another.
- **Why it's needed:** Input from files (CSVs) or users often comes as strings. You cannot perform math on a string "5" without casting it to an integer 5 first.
- **Functions:** int(), float(), str().

Data Types (Strings & Casting)

❑ Example:

```
raw_price = "0.99"    # String (from CSV)
# total = raw_price + 1 # ERROR: Cannot add int to str

# Correct approach: Casting
converted_price = float(raw_price)
total = converted_price + 1 # Output: 1.99

# Casting number to string for concatenation
msg = "The price is " + str(1.99)
```

Lists (The Fundamental Data Structure)

❑ Definition:

- A List is an ordered, mutable collection of items.
- **Heterogeneous:** A single list can contain mixed types (e.g., an integer, a string, and a float in the same list).

❑ Zero-Based Indexing:

- **Python lists are 0-indexed.** The first element is at position **0**, the second at **1**, etc..
- **Negative Indexing:** Python allows accessing elements from the end. **-1** is the last item, **-2** is the second to last.

❑ Slicing:

- Extracting a subset of the list using the syntax **list[start:stop]**.
- **Note:** The stop index is exclusive (**up to, but not including**).

Example 1 - Basic List Creation and Properties

```
# 1. Creating lists
empty_list = []
numbers = [1, 2, 3, 4, 5]
fruits = ["apple", "banana", "cherry"]
mixed = [1, "hello", 3.14, True, [1, 2, 3]] # Heterogeneous list

print(f"1. Empty list: {empty_list}")
# Output: 1. Empty list: []

print(f"2. Numbers: {numbers}")
# Output: 2. Numbers: [1, 2, 3, 4, 5]

print(f"3. Fruits: {fruits}")
# Output: 3. Fruits: ['apple', 'banana', 'cherry']

print(f"4. Mixed types: {mixed}")
# Output: 4. Mixed types: [1, 'hello', 3.14, True, [1, 2, 3]]
```

Example 1 - Basic List Creation and Properties (Cont.)

2. List length

```
print(f"\nList lengths:")
```

Output: List lengths:

```
print(f"  numbers length: {len(numbers)}")
```

Output: numbers length: 5

```
print(f"  fruits length: {len(fruits)}")
```

Output: fruits length: 3

```
print(f"  mixed length: {len(mixed)}")
```

Output: mixed length: 5

3. Type checking

```
print(f"\nType checking:")
```

Output: Type checking:

```
print(f"  type(numbers): {type(numbers)}")
```

Output: type(numbers): <class 'list'>

```
print(f"  isinstance(numbers, list): {isinstance(numbers, list)}")
```

Output: isinstance(numbers, list): True

Example 2 - Zero-Based and Negative Indexing

```
# Create a sample list
letters = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
print(f"List: {letters}")
# Output: List: ['A', 'B', 'C', 'D', 'E', 'F', 'G']

print("\n1. Zero-Based Indexing (Positive indices):")
# Output: 1. Zero-Based Indexing (Positive indices):

print(f"    letters[0] = '{letters[0]}' # First element")
# Output:    letters[0] = 'A' # First element
```

```
print("\n2. Negative Indexing (From the end):")
# Output: 2. Negative Indexing (From the end):

print(f"    letters[-1] = '{letters[-1]}' # Last element")
# Output:    letters[-1] = 'G' # Last element

print(f"    letters[-2] = '{letters[-2]}' # Second to last")
# Output:    letters[-2] = 'F' # Second to last
```

Example 3 - Slicing Basics

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(f"  numbers[2:5] = {numbers[2:5]}      # Elements 2, 3, 4 (stop at 5, exclusive)")
# Output:  numbers[2:5] = [2, 3, 4]      # Elements 2, 3, 4 (stop at 5, exclusive)

print(f"  numbers[:4]  = {numbers[:4]}      # First 4 elements (0 to 3)")
# Output:  numbers[:4]  = [0, 1, 2, 3]      # First 4 elements (0 to 3)

print(f"  numbers[6:]  = {numbers[6:]}      # From index 6 to end")
# Output:  numbers[6:]  = [6, 7, 8, 9]      # From index 6 to end

print(f"  numbers[:]   = {numbers[:]}      # Copy of entire list")
# Output:  numbers[:]   = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]      # Copy of entire list

print(f"  numbers[-4:]  = {numbers[-4:]}      # Last 4 elements")
# Output:  numbers[-4:] = [6, 7, 8, 9]      # Last 4 elements

print(f"  numbers[:-3]  = {numbers[:-3]}      # All except last 3")
# Output:  numbers[:-3] = [0, 1, 2, 3, 4, 5, 6]      # All except last 3

print(f"  numbers[-5:-2] = {numbers[-5:-2]} # Elements -5, -4, -3 (exclusive at -2)")
# Output:  numbers[-5:-2] = [5, 6, 7] # Elements -5, -4, -3 (exclusive at -2)
```

Example 3 - Slicing Basics (Cont.)

```
print("\n3. Slicing with step [start:stop:step]:")
# Output: 3. Slicing with step [start:stop:step]:

print(f"    numbers[::2] = {numbers[::2]}    # Every 2nd element")
# Output:    numbers[::2] = [0, 2, 4, 6, 8]    # Every 2nd element

print(f"    numbers[1::2] = {numbers[1::2]}    # Every 2nd element starting at 1")
# Output:    numbers[1::2] = [1, 3, 5, 7, 9]    # Every 2nd element starting at 1

print(f"    numbers[2:8:3] = {numbers[2:8:3]} # Elements 2, 5 (step 3)")
# Output:    numbers[2:8:3] = [2, 5] # Elements 2, 5 (step 3)

print(f"    numbers[::-1] = {numbers[::-1]}    # Reversed list")
# Output:    numbers[::-1] = [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]    # Reversed list
```

Example 4 - Common List Operations

```
fruits = ["apple", "banana"]
print(f"Initial list: {fruits}")
# Output: Initial list: ['apple', 'banana']

print("\n1. Adding elements:")
# Output: 1. Adding elements:

fruits.append("cherry")
print(f"    After append('cherry'): {fruits}")
# Output:    After append('cherry'): ['apple', 'banana', 'cherry']

fruits.insert(1, "orange")
print(f"    After insert(1, 'orange'): {fruits}")
# Output:    After insert(1, 'orange'): ['apple', 'orange', 'banana', 'cherry']

fruits.extend(["grape", "mango"])
print(f"    After extend(['grape', 'mango']): {fruits}")
# Output:    After extend(['grape', 'mango']): ['apple', 'orange', 'banana', 'cherry', 'grape', 'man
go']
```

Example 4 - Common List Operations (Cont.)

```
removed = fruits.pop() # Removes last
print(f"    After pop(): {fruits}, removed: '{removed}'")
# Output:    After pop(): ['apple', 'orange', 'banana', 'cherry', 'grape'], removed: 'mango'

removed = fruits.pop(1) # Removes at index 1
print(f"    After pop(1): {fruits}, removed: '{removed}'")
# Output:    After pop(1): ['apple', 'banana', 'cherry', 'grape'], removed: 'orange'

fruits.remove("banana")
print(f"    After remove('banana'): {fruits}")
# Output:    After remove('banana'): ['apple', 'cherry', 'grape']

print("\n3. Finding elements:")
# Output: 3. Finding elements:

print(f"    Index of 'apple': {fruits.index('apple')}")
# Output:    Index of 'apple': 0

print(f"    Count of 'apple': {fruits.count('apple')}")
# Output:    Count of 'apple': 1

print(f"    'mango' in fruits: {'mango' in fruits}")
# Output:    'mango' in fruits: False
```

Example 6 - Multi-dimensional Lists

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print("1. 2D List (Matrix):")  
# Output: 1. 2D List (Matrix):  
  
for row in matrix:  
    print(f"    {row}")  
# Output:    [1, 2, 3]  
# Output:    [4, 5, 6]  
# Output:    [7, 8, 9]  
  
print("\n2. Accessing elements:")  
# Output: 2. Accessing elements:  
  
print(f"    matrix[0][0] = {matrix[0][0]} # First row, first column")  
# Output:    matrix[0][0] = 1 # First row, first column  
  
print(f"    matrix[1][2] = {matrix[1][2]} # Second row, third column")  
# Output:    matrix[1][2] = 6 # Second row, third column
```

Example 6 - Multi-dimensional Lists (Cont.)

```
print(f"    matrix[-1][-1] = {matrix[-1][-1]}    # Last row, last column")
# Output:    matrix[-1][-1] = 9    # Last row, last column

print("\n3. Modifying elements:")
# Output: 3. Modifying elements:

matrix[1][1] = 99
print(f"    After matrix[1][1] = 99:")
# Output:    After matrix[1][1] = 99:

for row in matrix:
    print(f"    {row}")
# Output:    [1, 2, 3]
# Output:    [4, 99, 6]
# Output:    [7, 8, 9]
```

The Range Function (Sequence Generation)

❑ Definition:

- **range()** is an immutable sequence type that generates a sequence of integers.
- **Key Characteristic (Python 3 vs. 2):** In Python 3, range() does not return a list in memory. It returns a range object (a generator-like object) that produces numbers "on demand" (lazy evaluation).
- **Memory Efficiency:** A range(0, 1000000) takes up the same tiny amount of memory as range(0, 1) because it only stores the start, stop, and step values, calculating the numbers only when iterated over.

❑ Syntax:

- **range(stop):** Generates 0 up to (but not including) stop.
- **range(start, stop):** Generates from start up to (but not including) stop.
- **range(start, stop, step):** Increments by step. Step can be negative for reverse iteration.

Example 1 - Basic Syntax

```
print("1. range(stop):")
# Output: 1. range(stop):

r1 = range(5)
print(f"    range(5) = {r1}")
# Output:    range(5) = range(0, 5)

print(f"    list(range(5)) = {list(r1)}")
# Output:    list(range(5)) = [0, 1, 2, 3, 4]

r2 = range(2, 7)
print(f"    range(2, 7) = {r2}")
# Output:    range(2, 7) = range(2, 7)

print(f"    list(range(2, 7)) = {list(r2)}")
# Output:    list(range(2, 7)) = [2, 3, 4, 5, 6]
```

```
r3 = range(0, 10, 2)
print(f"    range(0, 10, 2) = {r3}")
# Output:    range(0, 10, 2) = range(0, 10, 2)

print(f"    list(range(0, 10, 2)) = {list(r3)}")
# Output:    list(range(0, 10, 2)) = [0, 2, 4, 6, 8]
```

```
print("\n4. Negative step (reverse):")
# Output: 4. Negative step (reverse):

r4 = range(10, 0, -1)
print(f"    range(10, 0, -1) = {r4}")
# Output:    range(10, 0, -1) = range(10, 0, -1)
```

The Enumerate Function (Pythonic Indexing)

❑ The Problem:

- When iterating over a list, we often need both the data (value) and its position (index).
- **The "Anti-Pattern" (C-Style):** Beginners often write for **i** in **range(len(my_list))**: to get the index. This is considered **non-Pythonic**, **harder to read, and slower**.

❑ The Solution: `enumerate()`

- **Definition:** `enumerate()` takes an **iterable** (like a list) and adds a counter to it, returning an **enumerate object** that **produces tuples in the format: (index, value)**.
- **Syntax:** `enumerate(iterable, start=0)`
- **Unpacking:** We typically "unpack" the tuple directly in the for loop definition: `for index, value in enumerate(...):`.

Example 1 - The Problem / Solution - Anti-Pattern

Range	Enumerate
Harder to read	Cleaner, more readable code
Need to manually index with fruits[i]	No manual indexing needed
Easy to make off-by-one errors	Less error-prone
Not Pythonic (doesn't follow Python conventions)	More Pythonic (follows Python conventions)
	Can customize start index

```
fruits = ["apple", "banana", "cherry", "date"]
for i in range(len(fruits)):
    print(f" Index {i}: {fruits[i]}")
# Output:   Index 0: apple
# Output:   Index 1: banana
# Output:   Index 2: cherry
# Output:   Index 3: date
```

```
fruits = ["apple", "banana", "cherry", "date"]
for index, fruit in enumerate(fruits):
    print(f" Index {index}: {fruit}")
# Output:   Index 0: apple
# Output:   Index 1: banana
# Output:   Index 2: cherry
# Output:   Index 3: date
```

Example 2 - Custom Start Index

```
students = ["Alice", "Bob", "Charlie"]
```

```
for i, student in enumerate(students):
```

```
    print(f" Student #{i}: {student}")
```

```
# Output: Student #0: Alice
```

```
# Output: Student #1: Bob
```

```
# Output: Student #2: Charlie
```

```
students = ["Alice", "Bob", "Charlie"]
```

```
for i, student in enumerate(students, start=1):
```

```
    print(f" {i}. {student}")
```

```
# Output: 1. Alice
```

```
# Output: 2. Bob
```

```
# Output: 3. Charlie
```

Lists of Lists (Representing Datasets)

Concept: The "Manual" DataFrame

- Before using libraries like Pandas, data analysis is done using "Lists of Lists."
- **Structure:** The outer list represents the dataset. Each inner list represents a single row (observation).

Accessing Nested Data:

- **Syntax:** `dataset[row_index][column_index]`
- The first index selects the list (row). The second index selects the item (column) inside that list.

Example 1 - Basic Structure - Manual DataFrame

```
# Create a dataset as a list of lists
# Each inner list is a row (observation)
dataset = [
    # [Name, Age, City, Salary]
    ["Alice", 30, "New York", 75000], # Row 0
    ["Bob", 25, "Los Angeles", 65000], # Row 1
    ["Charlie", 35, "Chicago", 85000], # Row 2
    ["Diana", 28, "Miami", 70000], # Row 3
]

for row in dataset:
    print(f" {row}")

# Output: ['Alice', 30, 'New York', 75000]
# Output: ['Bob', 25, 'Los Angeles', 65000]
# Output: ['Charlie', 35, 'Chicago', 85000]
# Output: ['Diana', 28, 'Miami', 70000]
```

Example 2 - Accessing Data - Single Elements

```
dataset = [  
    ["Alice", 30, "New York", 75000],  
    ["Bob", 25, "Los Angeles", 65000],  
    ["Charlie", 35, "Chicago", 85000],  
    ["Diana", 28, "Miami", 70000],  
]  
  
print(f"    dataset[0] = {dataset[0]}")  
# Output:    dataset[0] = ['Alice', 30, 'New York', 75000]  
  
print(f"    dataset[2] = {dataset[2]}")  
# Output:    dataset[2] = ['Charlie', 35, 'Chicago', 85000]  
  
print(f"    dataset[0][0] = {dataset[0][0]} # Row 0, Column 0 (Name)")  
# Output:    dataset[0][0] = Alice # Row 0, Column 0 (Name)  
  
print(f"    dataset[1][1] = {dataset[1][1]} # Row 1, Column 1 (Age)")  
# Output:    dataset[1][1] = 25 # Row 1, Column 1 (Age)  
  
print(f"    dataset[2][3] = {dataset[2][3]} # Row 2, Column 3 (Salary)")  
# Output:    dataset[2][3] = 85000 # Row 2, Column 3 (Salary)  
  
print(f"    dataset[3][2] = {dataset[3][2]} # Row 3, Column 2 (City)")  
# Output:    dataset[3][2] = Miami # Row 3, Column 2 (City)
```

Example 2 - Accessing Data - Single Elements (Cont.)

```
print(f"    dataset[-1] = {dataset[-1]}    # Last row")
```

```
# Output:    dataset[-1] = ['Diana', 28, 'Miami', 70000]    # Last row
```

```
print(f"    dataset[-1][-1] = {dataset[-1][-1]}    # Last row, last column")
```

```
# Output:    dataset[-1][-1] = 70000    # Last row, last column
```

```
print(f"    dataset[-2][-2] = {dataset[-2][-2]}    # Second last row, second last column")
```

```
# Output:    dataset[-2][-2] = Chicago    # Second last row, second last column
```

Example 3 - Modifying Data

```
dataset = [  
    ["Alice", 30, "New York", 75000],  
    ["Bob", 25, "Los Angeles", 65000],  
    ["Charlie", 35, "Chicago", 85000],  
]  
  
for row in dataset:  
    print(f" {row}")  
# Output:  ['Alice', 30, 'New York', 75000]  
# Output:  ['Bob', 25, 'Los Angeles', 65000]  
# Output:  ['Charlie', 35, 'Chicago', 85000]
```

```
dataset[0][1] = 31 # Change Alice's age  
print(f"  After dataset[0][1] = 31:")  
# Output:  After dataset[0][1] = 31:  
  
print(f"  Row 0: {dataset[0]}")  
# Output:  Row 0: ['Alice', 31, 'New York', 75000]  
  
dataset[1] = ["Robert", 26, "San Francisco", 68000]  
print(f"  After replacing Bob's row:")  
# Output:  After replacing Bob's row:  
  
print(f"  Row 1: {dataset[1]}")  
# Output:  Row 1: ['Robert', 26, 'San Francisco', 68000]
```

Example 3 - Modifying Data (Cont.)

```
dataset.insert(2, ["David", 32, "Boston", 80000])
print(f"    After insert(2, ['David', 32, 'Boston', 80000]):")
# Output:    After insert(2, ['David', 32, 'Boston', 80000]):

for i, row in enumerate(dataset):
    print(f"    Row {i}: {row}")
# Output:    Row 0: ['Alice', 31, 'New York', 75000]
# Output:    Row 1: ['Robert', 26, 'San Francisco', 68000]
# Output:    Row 2: ['David', 32, 'Boston', 80000]
# Output:    Row 3: ['Charlie', 35, 'Chicago', 85000]
# Output:    Row 4: ['Eve', 29, 'Seattle', 72000]

removed = dataset.pop(3) # Remove row 3 (Charlie)
print(f"    After pop(3) removed: {removed}")
# Output:    After pop(3) removed: ['Charlie', 35, 'Chicago', 85000]

print(f"    Remaining rows: {len(dataset)}")
# Output:    Remaining rows: 4
```

Example 5 - Basic Data Analysis

```
dataset = [
    # [Product, Units_Sold, Price, Region]
    ["Laptop", 120, 999.99, "North"],
    ["Mouse", 450, 29.99, "North"],
    ["Keyboard", 200, 79.99, "South"],
    ["Monitor", 85, 299.99, "North"],
    ["Headphones", 320, 149.99, "South"],
]

for row in dataset:
    print(f" {row}")
# Output:  ['Laptop', 120, 999.99, 'North']
# Output:  ['Mouse', 450, 29.99, 'North']
# Output:  ['Keyboard', 200, 79.99, 'South']
# Output:  ['Monitor', 85, 299.99, 'North']
# Output:  ['Headphones', 320, 149.99, 'South']

print("\n1. Calculate total units sold:")
# Output: 1. Calculate total units sold:

total_units = sum(row[1] for row in dataset)
print(f" Total units sold: {total_units}")
# Output: Total units sold: 1175
```

```
for row in dataset:
    product = row[0]
    units = row[1]
    price = row[2]
    revenue = units * price
    print(f" {product:12} → ${revenue:,.2f}")
# Output: Laptop → $119,998.80
# Output: Mouse → $13,495.50
# Output: Keyboard → $15,998.00
# Output: Monitor → $25,499.15
# Output: Headphones → $47,996.80

print("\n3. Calculate total revenue:")
# Output: 3. Calculate total revenue:

total_revenue = sum(row[1] * row[2] for row in dataset)
print(f" Total revenue: ${total_revenue:,.2f}")
# Output: Total revenue: $223,988.25
```

Dictionaries

❑ Concept: The Associative Array (Dictionary)

- **Definition:** A dictionary is an unordered, mutable collection of data values used to store data values like a map. Unlike lists that allow only a single value as an element, dictionaries hold a **key : value pair**.

❑ Key Characteristics:

- **Keys:** Must be unique and immutable (e.g., strings, numbers, tuples).
- **Values:** Can be any type (list, another dictionary, int, etc.).
- **Lookup Speed:** Dictionaries are implemented using **hash tables**, offering **$O(1)$** average time complexity for lookups, making them significantly faster than lists for searching.

Example 1 - Basic Dictionary Creation

```
empty_dict = {}
print(f"1. Empty dictionary: {empty_dict}")
# Output: 1. Empty dictionary: {}

# Dictionary with key-value pairs
student = {
    "name": "Alice",
    "age": 25,
    "major": "Computer Science",
    "gpa": 3.8
}
print(f"\n2. Student dictionary: {student}")
# Output: 2. Student dictionary: {'name': 'Alice', 'age': 25, 'major': 'Computer Science', 'gpa': 3.8}

# Using dict() constructor
product = dict(name="Laptop", price=999.99, in_stock=True)
print(f"\n3. Product dictionary: {product}")
# Output: 3. Product dictionary: {'name': 'Laptop', 'price': 999.99, 'in_stock': True}
```

Example 2 - Accessing Dictionary Elements

```
employee = {  
    "id": 12345,  
    "name": "John Doe",  
    "position": "Software Engineer",  
    "salary": 85000,  
    "skills": ["Python", "JavaScript", "SQL"]  
}
```

```
print(f"    employee['name'] = {employee['name']}")
```

```
# Output:    employee['name'] = John Doe
```

```
print(f"    employee['position'] = {employee['position']}")
```

```
# Output:    employee['position'] = Software Engineer
```

```
print("\n2. Access nested value:")
```

```
# Output: 2. Access nested value:
```

```
print(f"    employee['skills'][0] = {employee['skills'][0]}")
```

```
# Output:    employee['skills'][0] = Python
```

```
print("\n3. Using get() method (safer):")
```

```
# Output: 3. Using get() method (safer):
```

```
print(f"    employee.get('name') = {employee.get('name')}")
```

```
# Output:    employee.get('name') = John Doe
```

Example 2 - Accessing Dictionary Elements (Cont.)

```
print("\n4. Check if key exists:")
# Output: 4. Check if key exists:

print(f"    'name' in employee = {'name' in employee}")
# Output:    'name' in employee = True

print(f"    'address' in employee = {'address' in employee}")
# Output:    'address' in employee = False

print("\n5. Get all keys, values, and items:")
# Output: 5. Get all keys, values, and items:

print(f"    Keys: {list(employee.keys())}")
# Output:    Keys: ['id', 'name', 'position', 'salary', 'skills']

print(f"    Values: {list(employee.values())}")
# Output:    Values: [12345, 'John Doe', 'Software Engineer', 85000, ['Python', 'JavaScript', 'SQL']]

print(f"    Items: {list(employee.items())}")
# Output:    Items: [('id', 12345), ('name', 'John Doe'), ('position', 'Software Engineer'), ('salary', 85000), ('skills', ['Python', 'JavaScript', 'SQL'])]
```

Example 3 - Modifying Dictionaries

```
config = {
    "theme": "dark",
    "language": "English",
    "notifications": True
}

config["font_size"] = 14
print(f"    After config['font_size'] = 14: {config}")
# Output:    After config['font_size'] = 14: {'theme': 'dark', 'language': 'English', 'notification
s': True, 'font_size': 14}

print("\n3. Update existing value:")
# Output: 3. Update existing value:

config["theme"] = "light"
print(f"    After config['theme'] = 'light': {config}")
# Output:    After config['theme'] = 'light': {'theme': 'light', 'language': 'English', 'notificatio
ns': True, 'font_size': 14}

print("\n4. Update multiple values at once:")
# Output: 4. Update multiple values at once:

config.update({"language": "Spanish", "timezone": "EST"})
print(f"    After update(): {config}")
# Output:    After update(): {'theme': 'light', 'language': 'Spanish', 'notifications': True, 'font_
size': 14, 'timezone': 'EST'}
```

Example 3 - Modifying Dictionaries (Cont.)

```
# pop() removes and returns value
removed = config.pop("notifications")
print(f"    After pop('notifications'): {config}")
print(f"    Removed value: {removed}")
# Output:    After pop('notifications'): {'theme': 'light', 'language': 'Spanish', 'font_size': 14,
'timezone': 'EST'}
# Output:    Removed value: True

# del keyword
del config["font_size"]
print(f"    After del config['font_size']: {config}")
# Output:    After del config['font_size']: {'theme': 'light', 'language': 'Spanish', 'timezone': 'E
ST'}

# popitem() removes last inserted item
last_item = config.popitem()
print(f"    After popitem(): {config}")
print(f"    Last item removed: {last_item}")
# Output:    After popitem(): {'theme': 'light', 'language': 'Spanish'}
# Output:    Last item removed: ('timezone', 'EST')

config.clear()
print(f"    After clear(): {config}")
# Output:    After clear(): {}
```

Frequency Tables

□ Application: The Frequency Table

- **Logic:** A frequency table counts the occurrences of unique items in a dataset. In Python, we use a dictionary where the Key is the category (e.g., 'Games', 'Social') and the Value is the count (Integer).
- **Algorithm:** Iterate through the data; check if the item exists as a key. If yes, increment the value. If no, initialize the key with a value of 1.

Example 1 - Basic Frequency Table

```
app_usage = ["Games", "Social", "Games", "Productivity", "Social",
             "Games", "Entertainment", "Social", "Games", "Productivity"]
frequency = {}

for app in app_usage:
    if app in frequency:
        frequency[app] += 1 # Increment existing count
    else:
        frequency[app] = 1 # Initialize new category

print(f"\nFrequency table: {frequency}")
# Output: Frequency table: {'Games': 4, 'Social': 3, 'Productivity': 2, 'Entertainment': 1}

print("\nVisual representation:")
# Output: Visual representation:

print("  Category      Count")
# Output:   Category      Count

print("  -----      -----")
# Output:   -----      -----

for category, count in frequency.items():
    print(f" {category:12} {count:5}")
# Output:   Games          4
# Output:   Social         3
# Output:   Productivity    2
# Output:   Entertainment    1
```

Strings (Formatting, Cleaning, Slicing)

❑ String Formatting (Templating):

- **Concept:** Injecting variables into string literals to create dynamic text.
- **Method:** `.format()` replaces placeholder braces `{}` with arguments. It supports formatting instructions (e.g., `:.2f` for 2 decimal places).

❑ Data Cleaning:

- **Replacement:** The `.replace(old, new)` method is essential for cleaning "dirty" data, such as removing currency symbols or commas before mathematical conversion.

❑ Slicing:

- Strings behave like character arrays. You can extract substrings using `[start : stop]`. If start is omitted, it defaults to index 0.

Example 1 - Basic .format()

```
name = "Alice"
age = 30
text = "My name is {} and I am {} years old".format(name, age)
print(text) # Output: My name is Alice and I am 30 years old

price = 49.99
print("Price: ${:.2f}".format(price)) # Output: Price: $49.99

score = 0.875
print("Score: {:.1%}".format(score)) # Output: Score: 87.5%
```

Example 2 - Number Formatting

```
num = 1234.5678  
print("{:.2f}".format(num))      # Output: 1234.57  
print("{:,.2f}".format(num))     # Output: 1,234.57  
print("{:.0f}".format(num))      # Output: 1235  
  
value = 42  
print("{:06d}".format(value))    # Output: 000042
```

Example 3 - .replace() method

```
# Cleaning price strings
dirty_price = "$1,299.99"
clean_price = dirty_price.replace("$", "").replace(",", "")
print(clean_price) # Output: 1299.99

# Cleaning text data
text = "Hello***World!!!"
clean_text = text.replace("*", "").replace("!", "")
print(clean_text) # Output: HelloWorld
```

Example 4 - Multiple replacements

```
phone = "(123) 456-7890"
clean_phone = phone.replace("(", "").replace(")", "").replace(" ", "").replace("-", "")
print(clean_phone) # Output: 1234567890

# Convert to float for calculations
price_str = "$1,500.75"
price_num = float(price_str.replace("$", "").replace(",", ""))
print(price_num * 2) # Output: 3001.5
```

Example 5 - Basic slicing

```
text = "Hello World"

print(text[0:5])    # Output: Hello
print(text[6:])     # Output: World (from index 6 to end)
print(text[:5])     # Output: Hello (from start to index 4)
print(text[-5:])    # Output: World (last 5 characters)

# Get filename without extension
filename = "document.pdf"
name = filename[:-4] # Remove last 4 chars (.pdf)
print(name)         # Output: document
```

Example 5 - Step and negative indices

```
text = "Python"

print(text[::2])    # Output: Pto  (every 2nd character)
print(text[::-1])   # Output: nohtyP  (reverse string)
print(text[1:4])    # Output: yth
print(text[-3:])    # Output: hon  (last 3 characters)

# Get file extension
file = "image.jpg"
extension = file[-3:] # Get last 3 chars
print(extension)    # Output: jpg
```

Example 6 - Practical slicing examples

```
# Extract date parts
date = "2023-12-25"
year = date[0:4]      # Output: 2023
month = date[5:7]     # Output: 12
day = date[8:]        # Output: 25

# Get domain from email
email = "user@example.com"
domain = email[email.find("@")+1:] # Output: example.com

# Get initials from name
full_name = "John Doe"
initials = full_name[0] + full_name[full_name.find(" ")+1]
print(initials) # Output: JD
```

Control Flow (Logic & Loops)

❑ Conditional Statements (The Logic Gates):

- **If/Elif/Else:** These keywords control the flow of execution based on Boolean (True/False) conditions.
- **Comparisons:** == (equal), != (not equal), > (greater than). Note that == compares values, while = assigns values.
- **Boolean Logic:** conditions can be combined using and (both must be true) and or (at least one must be true).

❑ For Loops (Iteration):

- **Mechanism:** A for loop iterates over a sequence (such as a list or string), executing the block of code once for each item.
- **The Accumulator Pattern:** A common academic pattern where a variable (e.g., sum = 0) is initialized outside the loop and updated during each iteration to calculate a total.

Example 1 - Basic If/Elif/Else

```
# Temperature classification
temp = 25

if temp > 30:
    print("Hot")           # Output: (nothing, condition is False)
elif temp > 20:
    print("Warm")         # Output: Warm (condition is True)
else:
    print("Cold")

# Grade calculator
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"           # Output: grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(f"Score: {score}, Grade: {grade}") # Output: Score: 85, Grade: B
```

Example 2 - Comparisons and Boolean Logic

```
age = 25
has_license = True

# Simple comparisons
print(age == 25)    # Output: True
print(age != 25)    # Output: False
print(age > 18)     # Output: True

# Combined conditions with and/or
if age >= 18 and has_license:
    print("Can drive")    # Output: Can drive (both True)

if age < 13 or age > 65:
    print("Discount applies") # Output: (nothing, both False)

# Password validation
password = "Secure123"
if len(password) >= 8 and any(c.isdigit() for c in password):
    print("Valid password")    # Output: Valid password
```

Example 3 - Nested Conditions

```
# Discount calculator
membership = "gold"
purchase_amount = 200

if membership == "gold":
    if purchase_amount > 100:
        discount = 20      # Output: discount = 20
    else:
        discount = 10
elif membership == "silver":
    discount = 5
else:
    discount = 0

print(f"Discount: {discount}%") # Output: Discount: 20%
```

Example 4 - For Loop Basic Iteration

```
# Iterate over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
# Output: apple
#         banana
#         cherry

# Iterate over a string
word = "hello"
for letter in word:
    print(letter.upper(), end=" ") # Output: H E L L O
print()

# Iterate with range
for i in range(3):
    print(f"Loop {i}") # Output: Loop 0, Loop 1, Loop 2
```

Example 5 – Total / Avg. / Max

```
# Find average and maximum
scores = [85, 92, 78, 90, 88]
total = 0
max_score = scores[0] # Initialize with first value

for score in scores:
    total += score # Accumulate sum
    if score > max_score:
        max_score = score # Track maximum

average = total / len(scores)
print(f"Total: {total}")      # Output: Total: 433
print(f"Average: {average}")  # Output: Average: 86.6
print(f"Max: {max_score}")    # Output: Max: 92
```

Example 6 – String Accumulation

```
# Build a new string
names = ["Alice", "Bob", "Charlie"]
result = ""
for name in names:
    result = result + name + ", " # Accumulate strings
print(result) # Output: Alice, Bob, Charlie,

# Remove vowels
text = "hello world"
no_vowels = ""
for char in text:
    if char not in "aeiou":
        no_vowels += char # Output: no_vowels = "hll wrld"
```

Tuples (Data Integrity)

□ Tuples: The Immutable List

- **Definition:** Ordered sequences of elements, similar to lists, but defined with parentheses ().
- **Immutability:** Once created, elements cannot be added, removed, or changed.
- **Use Case:** Data Integrity. If you have a coordinate (lat, long), using a tuple ensures no code can accidentally overwrite the latitude. They are also slightly faster than lists.

Example 1 - Basic Tuple Operations

```
# Creating tuples
coordinates = (40.7128, -74.0060) # Latitude, Longitude
rgb_color = (255, 0, 0)          # Red color
person = ("Alice", 30, "Engineer")

print(f"Coordinates: {coordinates}") # Output: (40.7128, -74.0060)
print(f"RGB Color: {rgb_color}")     # Output: (255, 0, 0)
print(f"Person: {person}")           # Output: ('Alice', 30, 'Engineer')

# Accessing elements (like lists)
print(f"\nLatitude: {coordinates[0]}") # Output: 40.7128
print(f"Longitude: {coordinates[1]}")  # Output: -74.0060
print(f"Red: {rgb_color[0]}")          # Output: 255

# Tuple unpacking
name, age, job = person
print(f"\nUnpacked: {name}, {age}, {job}") # Output: Alice, 30, Engineer
```

Example 2 - Immutability Demonstration

```
# Try to modify a tuple - this will fail
months = ("Jan", "Feb", "Mar", "Apr")

print(f"Original: {months}") # Output: ('Jan', 'Feb', 'Mar', 'Apr')

# This works - accessing elements
print(f"First month: {months[0]}") # Output: Jan
print(f"Last month: {months[-1]}") # Output: Apr

# This will FAIL - tuples are immutable
try:
    months[0] = "January" # Trying to change element
except TypeError as e:
    print(f"\nError when trying to modify: {e}")
    # Output: 'tuple' object does not support item assignment

# This will also FAIL
try:
    months.append("May") # Trying to add element
except AttributeError as e:
    print(f"Error when trying to append: {e}")
    # Output: 'tuple' object has no attribute 'append'
```

Example 3 - When to Use Tuples (Data Integrity)

```
# Database configuration - should never change
DATABASE_CONFIG = ("localhost", 5432, "my_database", "readonly_user")

print("Database Configuration:")
print(f"Host: {DATABASE_CONFIG[0]}")      # Output: localhost
print(f"Port: {DATABASE_CONFIG[1]}")      # Output: 5432
print(f"Database: {DATABASE_CONFIG[2]}")  # Output: my_database
print(f"User: {DATABASE_CONFIG[3]}")      # Output: readonly_user

# Attempt to change config (will fail, protecting data integrity)
try:
    DATABASE_CONFIG[1] = 3306 # Try to change port
except TypeError:
    print("\nConfig is protected! Cannot change database settings.")
    # Output: Config is protected! Cannot change database settings.

# Function returning multiple values (tuples are perfect for this)
def get_user_info(user_id):
    # Simulating database lookup
    return ("Alice Johnson", "alice@email.com", "555-1234")

name, email, phone = get_user_info(123)
print(f"\nUser Info: {name}, {email}, {phone}")
# Output: User Info: Alice Johnson, alice@email.com, 555-1234
```

Example - Tuple vs List Performance

```
import time

# Create large list and tuple
large_data = list(range(1000000))
large_tuple = tuple(range(1000000))

# Timing tuple creation
start = time.time()
t = tuple(range(1000000))
tuple_time = time.time() - start

# Timing list creation
start = time.time()
l = list(range(1000000))
list_time = time.time() - start

print(f"Tuple creation time: {tuple_time:.6f} seconds")
print(f"List creation time: {list_time:.6f} seconds")
# Output example:
# Tuple creation time: 0.021345 seconds
# List creation time: 0.024567 seconds
# (Tuples are slightly faster)
```

Sets

❑ Sets: The Unique Collection

- **Definition:** Unordered collections of unique elements, defined with curly braces {} or set().
- **Theory:** Sets are implemented using Hash Tables.
- **Use Case:** Instant De-duplication. **Converting a list to a set automatically removes all duplicate values.**

Example 1 - Basic Set Operations

```
# Creating sets
colors = {"red", "green", "blue", "red"} # Duplicates removed automatically
numbers = set([1, 2, 3, 3, 2, 1])      # From list with duplicates
empty_set = set()                     # Note: {} creates empty dict!

print(f"Colors: {colors}")             # Output: {'red', 'green', 'blue'} (no duplicates)
print(f"Numbers: {numbers}")           # Output: {1, 2, 3} (no duplicates)
print(f"Empty set: {empty_set}")       # Output: set()

# Adding elements
colors.add("yellow")
print(f"\nAfter adding yellow: {colors}") # Output: {'red', 'green', 'blue', 'yellow'}

# Adding duplicate - nothing happens
colors.add("red")
print(f"After adding 'red' again: {colors}") # Output: Same as before

# Removing elements
colors.remove("green")
print(f"After removing green: {colors}") # Output: {'red', 'blue', 'yellow'}
```

Example 2 - Instant De-duplication

```
# List with many duplicates
duplicate_data = ["apple", "banana", "apple", "orange", "banana", "apple", "grape"]

print(f"Original list: {duplicate_data}")
print(f"Length: {len(duplicate_data)}") # Output: 7

# Convert to set to remove duplicates
unique_items = set(duplicate_data)
print(f"\nUnique items (set): {unique_items}")
print(f"Length: {len(unique_items)}") # Output: 4

# Convert back to list if needed
unique_list = list(unique_items)
print(f"Unique items (list): {unique_list}")
print(f"Sorted: {sorted(unique_list)}")
# Output: ['apple', 'banana', 'grape', 'orange']
```

Example 3 - Real-World De-duplication

```
# Email list with duplicates
raw_emails = [
    "alice@example.com",
    "bob@example.com",
    "alice@example.com", # Duplicate
    "charlie@example.com",
    "bob@example.com",   # Duplicate
    "diana@example.com",
    "alice@example.com", # Duplicate again
]

# Remove duplicates instantly with set
unique_emails = set(raw_emails)
print(f"Raw email count: {len(raw_emails)}")      # Output: 7
print(f"Unique email count: {len(unique_emails)}") # Output: 4
print(f"Unique emails: {unique_emails}")
# Output: {'alice@example.com', 'bob@example.com', 'charlie@example.com', 'diana@example.com'}
```

Example 3 - Real-World De-duplication (Cont.)

```
# Find common customers between two stores
store_a_customers = {"customer1@email.com", "customer2@email.com", "customer3@email.com"}
store_b_customers = {"customer2@email.com", "customer3@email.com", "customer4@email.com"}

common_customers = store_a_customers.intersection(store_b_customers)
print(f"\nCustomers who shop at both stores: {common_customers}")
# Output: {'customer2@email.com', 'customer3@email.com'}
```

Example 4 - Membership Testing (Fast Lookup)

```
# Sets are optimized for membership testing
large_list = list(range(1000000))
large_set = set(range(1000000))

# Test number to find
test_number = 999999

# List membership test (slower - O(n))
import time
start = time.time()
in_list = test_number in large_list
list_time = time.time() - start

# Set membership test (faster - O(1))
start = time.time()
in_set = test_number in large_set
set_time = time.time() - start

print(f"List lookup: {list_time:.6f} seconds")
print(f"Set lookup: {set_time:.6f} seconds")
print(f"Set is {list_time/set_time:.0f}x faster!")
# Output example:
# List lookup: 0.008765 seconds
# Set lookup: 0.000003 seconds
# Set is 2921x faster!
```

```
# Practical example: Filter valid items
valid_codes = {"ABC123", "DEF456", "GHI789", "JKL012"}
user_inputs = ["ABC123", "XYZ999", "DEF456", "123456"]

print("\nValidating codes:")
for code in user_inputs:
    if code in valid_codes: # Fast lookup
        print(f"✓ {code} is valid")
    else:
        print(f"✗ {code} is invalid")

# Output:
# ✓ ABC123 is valid
# ✗ XYZ999 is invalid
# ✓ DEF456 is valid
# ✗ 123456 is invalid
```

Example 5 - Tuple-Set Combination

```
# Tuples can be elements of sets (because they're immutable)
# This is useful for storing unique pairs/coordinates

# Unique coordinate pairs
coordinates_set = {
    (40.7128, -74.0060), # NYC
    (34.0522, -118.2437), # LA
    (40.7128, -74.0060), # NYC again (duplicate)
    (51.5074, -0.1278), # London
}

print(f"Unique coordinates: {coordinates_set}")
print(f"Number of unique locations: {len(coordinates_set)}") # Output: 3 (NYC duplicate removed)

# Check if location exists
nyc = (40.7128, -74.0060)
chicago = (41.8781, -87.6298)

print(f"\nIs NYC in set? {nyc in coordinates_set}") # Output: True
print(f"Is Chicago in set? {chicago in coordinates_set}") # Output: False
```

Example 5 - Tuple-Set Combination (Cont.)

```
# List of tuples with duplicates
student_grades = [
    ("Alice", "Math", "A"),
    ("Bob", "Science", "B"),
    ("Alice", "Math", "A"), # Duplicate
    ("Charlie", "History", "C"),
    ("Bob", "Science", "B"), # Duplicate
]

# Remove duplicate records
unique_grades = set(student_grades)
print(f"\nUnique grade records: {unique_grades}")
# Output: {('Alice', 'Math', 'A'), ('Bob', 'Science', 'B'), ('Charlie', 'History', 'C')}
```

Example 6 - Frozen Sets (Immutable Sets)

```
# Frozen sets are immutable sets
valid_states = frozenset(["CA", "NY", "TX", "FL", "IL"])

print(f"Valid states: {valid_states}")
print(f"Is CA valid? {'CA' in valid_states}") # Output: True
print(f"Is ZZ valid? {'ZZ' in valid_states}") # Output: False

# Frozen sets can't be modified
try:
    valid_states.add("GA")
except AttributeError as e:
    print(f"Cannot modify frozen set: {e}")
    # Output: 'frozenset' object has no attribute 'add'

# Use case: constant lookup table
COLORS = frozenset(["red", "green", "blue", "yellow", "purple"])

user_color = "pink"
if user_color in COLORS:
    print(f"{user_color} is a valid color")
else:
    print(f"{user_color} is not a valid color") # Output: pink is not a valid color
```

Example 7 - Set Operations (Union, Intersection, Diff)

```
# Two sets of data
python_devs = {"Alice", "Bob", "Charlie", "Diana"}
js_devs = {"Bob", "Diana", "Eve", "Frank"}

print(f"Python devs: {python_devs}")
print(f"JS devs: {js_devs}")

# Union (all developers)
all_devs = python_devs.union(js_devs)
print(f"\nAll devs (union): {all_devs}")
# Output: {'Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'}
```

```
# Intersection (devs who know both)
both_languages = python_devs.intersection(js_devs)
print(f"Know both (intersection): {both_languages}")
# Output: {'Bob', 'Diana'}

# Difference (Python-only devs)
python_only = python_devs.difference(js_devs)
print(f"Python only (difference): {python_only}")
# Output: {'Alice', 'Charlie'}

# Symmetric difference (devs who know only one language)
only_one = python_devs.symmetric_difference(js_devs)
print(f"Know only one (symmetric difference): {only_one}")
# Output: {'Alice', 'Charlie', 'Eve', 'Frank'}
```

Functions (Modular Programming)

- ❑ **Definition:** A function is a block of organized, reusable code that performs a single, related action. It provides better modularity for your application and a high degree of code reuse.
- ❑ **Syntax:**
 - **def:** Keyword to define the function.
 - **parameters:** Inputs the function expects (enclosed in parentheses).
 - **return:** The statement that exits the function and passes a value back to the caller.
- ❑ **Scope:** Variables defined inside a function are **local**; **they cannot be accessed outside the function.**

Functions (Modular Programming)

❑ Example:

```
def square(number):           # 'number' is a parameter
    return number ** 2        # Returns the square

result = square(5)            # Call with argument 5
# Output: 25 [cite: 124, 135]
```

Advanced Arguments & Return Values

❑ **Positional vs. Keyword Arguments:**

- **Positional:** Arguments passed in order (e.g., `subtract(10, 5)`).
- **Keyword:** Arguments passed by name, allowing order independence (e.g., `subtract(b=5, a=10)`).

❑ **Default Arguments:** Parameters can assume a default value **if no argument** is provided during the call, enhancing flexibility.

❑ **Multiple Return Values:** Python functions can return multiple values as a tuple, which can be unpacked into separate variables.

Example1 - Basic Function

```
def subtract(a, b):  
    return a - b  
  
print("Positional arguments (order matters):")  
result1 = subtract(10, 5)  
print(f"subtract(10, 5) = {result1}")  
  
print("\nKeyword arguments (order doesn't matter):")  
result2 = subtract(b=5, a=10)  
print(f"subtract(b=5, a=10) = {result2}")  
  
result3 = subtract(a=15, b=3)  
print(f"subtract(a=15, b=3) = {result3}")
```

Positional arguments (order matters):

`subtract(10, 5) = 5`

Keyword arguments (order doesn't matter):

`subtract(b=5, a=10) = 5`

`subtract(a=15, b=3) = 12`

Example 2 - Common Error - Positional After Keyword

```
def multiply(a, b, c=1):  
    return a * b * c  
  
print("Valid - positional before keyword:")  
result1 = multiply(2, 3, c=4)  
print(f"multiply(2, 3, c=4) = {result1}")  
  
print("\nValid - all keyword arguments:")  
result2 = multiply(c=5, a=2, b=3)  
print(f"multiply(c=5, a=2, b=3) = {result2}")  
  
print("\nTrying invalid - positional after keyword:")  
try:  
    result3 = multiply(a=2, 3, c=4)  
    print(f"This won't print")  
except SyntaxError as e:  
    print(f"SyntaxError: Positional argument cannot appear after keyword argument")
```

Example 2 - Common Error - Positional After Keyword

Valid - positional before keyword:

```
multiply(2, 3, c=4) = 24
```

Valid - all keyword arguments:

```
multiply(c=5, a=2, b=3) = 30
```

Trying invalid - positional after keyword:

```
SyntaxError: Positional argument cannot appear after keyword argument
```

List Comprehensions (Pythonic Iteration)

❑ Concept: Syntactic Sugar

- **Definition:** List comprehensions provide a concise way to create lists. They are syntactically more compact and often faster than standard for loops because the iteration is optimized in C (the language Python is written in).
- **Structure:** [expression for item in iterable if condition]

❑ The Transformation: We move from an Imperative style (telling the computer how to build the list step-by-step) to a Declarative style (describing what we want the list to look like).

Example 1 - Basic Transformation - For Loop to List Comprehension

```
# For Loop (Imperative style - "how to build it")
numbers = [1, 2, 3, 4, 5]
squared_loop = []
for n in numbers:
    squared_loop.append(n ** 2)
print(squared_loop) # Output: [1, 4, 9, 16, 25]

# List Comprehension (Declarative style - "what we want")
squared_comp = [n ** 2 for n in numbers]
print(squared_comp) # Output: [1, 4, 9, 16, 25]
```

Example 2: With Condition Filter

```
# Filter even numbers - For Loop
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
evens_loop = []
for n in numbers:
    if n % 2 == 0:
        evens_loop.append(n)
print(evens_loop) # Output: [2, 4, 6, 8]

# List Comprehension with condition
evens_comp = [n for n in numbers if n % 2 == 0]
print(evens_comp) # Output: [2, 4, 6, 8]
```

Example 3 - Transform and Filter

```
# Square only even numbers - For Loop
numbers = [1, 2, 3, 4, 5]
squared_evens_loop = []
for n in numbers:
    if n % 2 == 0:
        squared_evens_loop.append(n ** 2)
print(squared_evens_loop) # Output: [4, 16]

# List Comprehension with both transformation and filter
squared_evens_comp = [n ** 2 for n in numbers if n % 2 == 0]
print(squared_evens_comp) # Output: [4, 16]
```

Example 4 - String Processing

```
words = ["hello", "world", "python", "code"]

# Convert to uppercase - For Loop
uppercase_loop = []
for word in words:
    uppercase_loop.append(word.upper())
print(uppercase_loop) # Output: ['HELLO', 'WORLD', 'PYTHON', 'CODE']

# List Comprehension
uppercase_comp = [word.upper() for word in words]
print(uppercase_comp) # Output: ['HELLO', 'WORLD', 'PYTHON', 'CODE']

# Only words longer than 5 characters
long_words = [word for word in words if len(word) > 5]
print(long_words) # Output: ['python']
```

Example 5 - Nested Comprehension

```
# Create a matrix - For Loop
matrix_loop = []
for i in range(3):
    row = []
    for j in range(3):
        row.append(i * j)
    matrix_loop.append(row)
print(matrix_loop) # Output: [[0, 0, 0], [0, 1, 2], [0, 2, 4]]

# Nested List Comprehension
matrix_comp = [[i * j for j in range(3)] for i in range(3)]
print(matrix_comp) # Output: [[0, 0, 0], [0, 1, 2], [0, 2, 4]]
```

Example 6 - Real-World Applications

```
# Extract numbers from a string
text = "The price is $29.99 and quantity is 5"
numbers = [char for char in text if char.isdigit()]
print(numbers) # Output: ['2', '9', '9', '9', '5']

# Convert temperatures
fahrenheit = [32, 68, 86, 104]
celsius = [(temp - 32) * 5/9 for temp in fahrenheit]
print(celsius) # Output: [0.0, 20.0, 30.0, 40.0]

# Process files - get file extensions
files = ["report.pdf", "image.jpg", "data.csv", "notes.txt"]
extensions = [file.split(".")[-1] for file in files]
print(extensions) # Output: ['pdf', 'jpg', 'csv', 'txt']
```

Example 7 - Performance Comparison

```
# Timing comparison
n = 1_000_000

# For Loop
start = time.time()
result_loop = []
for i in range(n):
    result_loop.append(i ** 2)
time_loop = time.time() - start

# List Comprehension
start = time.time()
result_comp = [i ** 2 for i in range(n)]
time_comp = time.time() - start

print(f"For Loop time: {time_loop:.4f} seconds")
print(f"List Comprehension time: {time_comp:.4f} seconds")
print(f"Speed improvement: {time_loop/time_comp:.1f}x faster")

# Output (example):
# For Loop time: 0.0732 seconds
# List Comprehension time: 0.0381 seconds
# Speed improvement: 1.9x faster
```

Example 8 - Condition at Beginning (if-else in expression)

```
numbers = [1, 2, 3, 4, 5]

# Categorize numbers - For Loop
categories_loop = []
for n in numbers:
    if n % 2 == 0:
        categories_loop.append("even")
    else:
        categories_loop.append("odd")
print(categories_loop) # Output: ['odd', 'even', 'odd', 'even', 'odd']

# List Comprehension with if-else
categories_comp = ["even" if n % 2 == 0 else "odd" for n in numbers]
print(categories_comp) # Output: ['odd', 'even', 'odd', 'even', 'odd']
```

Example 9 - Multiple Conditions

```
numbers = [10, 15, 20, 25, 30, 35, 40]

# Multiple conditions - For Loop
filtered_loop = []
for n in numbers:
    if n % 10 == 0 and n > 15:
        filtered_loop.append(n)
print(filtered_loop) # Output: [20, 30, 40]

# List Comprehension with multiple conditions
filtered_comp = [n for n in numbers if n % 10 == 0 and n > 15]
print(filtered_comp) # Output: [20, 30, 40]
```

Error Handling (Robustness)

❑ Concept: Graceful Failure

- **Reality of Data:** In data science, datasets are rarely perfect. They contain missing values, "N/A" strings in number columns, or typos.
- **Exceptions:** When Python encounters an error (like trying to divide by zero), it raises an "Exception." If unhandled, the program crashes immediately.

❑ The Mechanism (try / except):

- **Try Block:** Code that might cause an error.
- **Except Block:** Code that runs only if an error occurs. This prevents the crash and allows the program to log the error and continue.
- **Philosophy:** "It is easier to ask for forgiveness than permission".

Common Python Exception Types

❑ The Exception Hierarchy:

- In Python, all exceptions are objects that inherit from the **BaseException** class.
- We typically catch errors inheriting from Exception (the base class for all non-fatal code errors).
- **Best Practice:** Always catch specific exceptions (e.g., **ValueError**) rather than a blanket except Exception:. This prevents masking unexpected bugs.

1. Logic & Math Errors:

- **ZeroDivisionError:** Raised when the second argument of a division or modulo operation is zero.
 - **Example:** `x = 10 / 0`

Common Python Exception Types

2. Data Structure Errors (Lookup):

- **IndexError:** Raised when a sequence subscript (index) is out of range.
 - **Example:** `Accessing my_list[10]` when the list only has 3 items.
- **KeyError:** Raised when a dictionary key is not found in the set of existing keys.
 - **Example:** `my_dict['unknown_key']`

Common Python Exception Types

3. Type & Value Errors (Most Common):

- **TypeError:** Raised when an operation or function is applied to an object of inappropriate type.
 - **Example: Trying to add a string and an integer ("score: " + 10).**
- **ValueError:** Raised when an operation receives an argument that has the right type but an inappropriate value.
 - **Example: int("hello") (String is the right type, but "hello" is not a number).**

Common Python Exception Types

4. Input/Output (I/O) Errors:

- **FileNotFoundError:** Raised when a file or directory is requested but doesn't exist.
- **IOError:** General input/output failures (often related to permissions or disk issues).

Example 1 - Handling Invalid Data Conversion

```
# Real-world data with mixed types
dirty_data = ["25", "42", "N/A", "18", "unknown", "30"]

print("Cleaning dirty data:")
clean_numbers = []
for item in dirty_data:
    try:
        number = int(item) # This might fail
        clean_numbers.append(number)
        print(f"✓ Converted '{item}' to {number}")
    except ValueError:
        print(f"✗ Could not convert '{item}' to integer - skipping")

print(f"\nClean data: {clean_numbers}")

# Output:
# ✓ Converted '25' to 25
# ✓ Converted '42' to 42
# ✗ Could not convert 'N/A' to integer - skipping
# ✓ Converted '18' to 18
# ✗ Could not convert 'unknown' to integer - skipping
# ✓ Converted '30' to 30
```

Example 2 - Multiple Exception Types

```
# Handle different types of errors
data = ["100", "50", "0", "invalid", "25"]

print("Processing with multiple exception types:")
for item in data:
    try:
        value = int(item)
        result = 1000 / value
        print(f"1000 / {value} = {result}")
    except ValueError:
        print(f"Error: '{item}' is not a valid number")
    except ZeroDivisionError:
        print(f"Error: Cannot divide by zero (value: {item})")

# Output:
# 1000 / 100 = 10.0
# 1000 / 50 = 20.0
# Error: Cannot divide by zero (value: 0)
# Error: 'invalid' is not a valid number
# 1000 / 25 = 40.0
```

Example 3 - Real-World Data Cleaning

```
# Clean dataset with currency values and missing data
dirty_sales = ["$1,299.99", "N/A", "899.50", "$2,500", "invalid", "750"]

print("Cleaning sales data:")
clean_sales = []
for price in dirty_sales:
    try:
        # Remove $ and commas, convert to float
        cleaned = price.replace("$", "").replace(",", "")
        numeric_price = float(cleaned)
        clean_sales.append(numeric_price)
        print(f"✓ Cleaned: {price} → ${numeric_price:.2f}")
    except (ValueError, AttributeError):
        print(f"✗ Could not process: {price}")

print(f"\nValid prices: {clean_sales}")
print(f"Total sales: ${sum(clean_sales):.2f}")

# Output:
# ✓ Cleaned: $1,299.99 → $1299.99
# ✗ Could not process: N/A
# ✓ Cleaned: 899.50 → $899.50
# ✓ Cleaned: $2,500 → $2500.00
# ✗ Could not process: invalid
# ✓ Cleaned: 750 → $750.00
#
# Valid prices: [1299.99, 899.5, 2500.0, 750.0]
# Total sales: $5449.49
```

Raising Exceptions (The raise Statement)

❑ Concept: Explicit Error Signaling

- **Definition:** While Python raises exceptions automatically when it encounters errors (like dividing by zero), the raise keyword allows the programmer to force a specific exception to occur.
- **The "Fail Fast" Philosophy:** It is better for a program to crash immediately with a clear error message than to continue running with corrupt or invalid data (which leads to silent bugs later).

❑ Syntax:

- Use the raise keyword followed by an Exception class instance.
- **Format:** raise ExceptionType("Descriptive Error Message")

Raising Exceptions (The raise Statement)

❑ Why Raise Manually? (Use Cases)

- **Input Validation:** To reject invalid arguments passed to a function (e.g., **preventing a negative age or a price of \$0**).
- **Abstract Methods:** To force subclasses to implement a specific method (**using NotImplementedError**).
- **Business Logic Enforcement:** **To stop a transaction if the user has insufficient funds.**

Example 1 - Basic Input Validation

```
def calculate_discount(price, discount_percent):  
    """  
    Calculate price after discount.  
    Raises ValueError for invalid inputs.  
    """  
    # Validate inputs explicitly  
    if price <= 0:  
        raise ValueError(f"Price must be positive, got: ${price}")  
  
    if discount_percent < 0:  
        raise ValueError(f"Discount cannot be negative, got: {discount_percent}%")  
  
    if discount_percent > 100:  
        raise ValueError(f"Discount cannot exceed 100%, got: {discount_percent}%")  
  
    # Calculate if all validations pass  
    discounted_price = price * (1 - discount_percent / 100)  
    return discounted_price
```

Example 1 - Basic Input Validation (Cont.)

```
# Test valid and invalid inputs
test_cases = [
    (100, 20),    # Valid: $100 with 20% discount
    (50, 0),      # Valid: $50 with 0% discount
    (-50, 10),    # Invalid: Negative price
    (100, -10),   # Invalid: Negative discount
    (100, 120),   # Invalid: Discount > 100%
]

print("Testing discount calculations:")
for price, discount in test_cases:
    try:
        result = calculate_discount(price, discount)
        print(f"✓ Price: ${price}, Discount: {discount}% → ${result:.2f}")
    except ValueError as e:
        print(f"✗ Price: ${price}, Discount: {discount}% → ERROR: {e}")
```

Output:

Testing discount calculations:

✓ Price: \$100, Discount: 20% → \$80.00

✓ Price: \$50, Discount: 0% → \$50.00

✗ Price: \$-50, Discount: 10% → ERROR: Price must be positive, got: \$-50

✗ Price: \$100, Discount: -10% → ERROR: Discount cannot be negative, got: -10%

✗ Price: \$100, Discount: 120% → ERROR: Discount cannot exceed 100%, got: 120%

The Assert Statement (Internal Self-Checks)

❑ Concept: Debugging Logic

- **Definition:** An assert statement is a logic check that you turn on or off. It tests a condition, and if the condition is False, it raises an AssertionError.
- **Purpose:** It is used to catch developer errors (**bugs in the logic**), **not user errors**. It declares "at this point in the code, this condition must be true; if not, the code is broken."
- **Optimization:** When Python is run in optimized mode (using the -O flag), **all assert statements are ignored and removed from execution**. **Never use assert for validating user input (e.g., passwords), because it can be disabled!**

The Assert Statement (Internal Self-Checks)

❑ Syntax:

- assert condition, "Optional Error Message"
- If condition is **True** → **Nothing happens (Code continues)**.
- If condition is **False** → **Raises AssertionError with the message**.

❑ Use Cases:

- **Invariants:** Checking conditions that should always remain true (e.g., "The list length must remain 52 after shuffling").
- **Unreachable Code:** Placing assert False in a block of code that should logically never be executed.
- **Post-Conditions:** Verifying that a function returned a sensible result before passing it on.

Example 1 - Basic Assert Usage

```
def calculate_average(numbers):  
    """  
    Calculate average of a list of numbers.  
    Uses assert for debugging invariants.  
    """  
  
    # Pre-condition: input must be a list  
    assert isinstance(numbers, list), "Input must be a list"  
  
    # Pre-condition: list cannot be empty  
    assert len(numbers) > 0, "Cannot calculate average of empty list"  
  
    # Pre-condition: all elements must be numbers  
    assert all(isinstance(n, (int, float)) for n in numbers), \  
        "All elements must be numbers"  
  
    # Calculate average  
    total = sum(numbers)  
    count = len(numbers)  
    average = total / count  
  
    # Post-condition: average should be between min and max  
    assert min(numbers) <= average <= max(numbers), \  
        f"Average {average} not between {min(numbers)} and {max(numbers)}"  
  
    return average
```

Example 1 - Basic Assert Usage (Cont.)

```
# Test with valid input
print("Test 1: Valid input")
numbers = [10, 20, 30, 40, 50]
result = calculate_average(numbers)
print(f"Average of {numbers} = {result}") # Output: 30.0

print("\nTest 2: Empty list (should fail)")
try:
    calculate_average([])
except AssertionError as e:
    print(f"AssertionError: {e}")

print("\nTest 3: Non-numeric elements (should fail)")
try:
    calculate_average([10, 20, "thirty", 40])
except AssertionError as e:
    print(f"AssertionError: {e}")

"""
Output:
Test 1: Valid input
Average of [10, 20, 30, 40, 50] = 30.0

Test 2: Empty list (should fail)
AssertionError: Cannot calculate average of empty list

Test 3: Non-numeric elements (should fail)
AssertionError: All elements must be numbers
"""
```

File Handling (Input/Output)

❑ The Concept:

- File handling allows Python to interact with external files (text, CSV, logs) on the disk.
- **The Process: Open → Process (Read/Write) → Close.**

1. Access Modes (The mode parameter):

- **'r' (Read):** Default. Opens for reading. Returns error if file does not exist.
- **'w' (Write):** Opens for writing. Creates a new file or truncates (wipes) an existing file.
- **'a' (Append):** Opens for writing. Data is added to the end. Preserves existing data.
- **'r+' (Read/Write):** Opens for both reading and writing.

File Handling (Input/Output)

2. Reading Methods (Memory Implications):

- **.read():** Reads the entire file into a single string. (Fast, but dangerous for huge files).
- **.readline():** Reads just one line at a time. (Memory efficient).
- **.readlines():** Reads the entire file and returns a list of strings (one per line).

3. The Golden Rule (Context Managers):

- **Always use the with statement. It automatically handles the .close() call, even if your code crashes during processing.**

Example 1 - Basic File Operations with Different Modes

```
# 1. Write mode ('w') - Creates or overwrites file
print("\n1. Write Mode ('w'):")
with open("example.txt", "w") as file:
    file.write("Hello World!\n")
    file.write("This is line 2.\n")
    file.write("Python is awesome!\n")
print("Created 'example.txt' with 3 lines")

# 2. Read mode ('r') - Default mode
print("\n2. Read Mode ('r'):")
with open("example.txt", "r") as file:
    content = file.read()
    print("Full content:")
    print(content)

# Output:
# Hello World!
# This is line 2.
# Python is awesome!
```

```
# 3. Append mode ('a') - Adds to existing file
print("\n3. Append Mode ('a'):")
with open("example.txt", "a") as file:
    file.write("This line was appended.\n")
    file.write("So was this one!\n")
print("Appended 2 more lines to 'example.txt'")

# Read again to see appended content
with open("example.txt", "r") as file:
    print("Updated content:")
    print(file.read())

# Output includes the appended lines

# 4. Read/Write mode ('r+')
print("\n4. Read/Write Mode ('r+'):")
with open("example.txt", "r+") as file:
    # Read first
    content = file.read()
    print("Original content length:", len(content))

    # Write at current position (end of file after read)
    file.write("Added with r+ mode.\n")
print("Added line using r+ mode")
```

Example 2 - Different Reading Methods

```
# Method 1: .read() - Entire file at once
print("\n1. Using .read() (entire file):")
with open("data.txt", "r") as file:
    all_content = file.read()
    print(f"Type: {type(all_content)}") # Output: <class 'str'>
    print(f"Length: {len(all_content)} characters")
    print("First 50 chars:", all_content[:50])
# Warning: .read() loads entire file into memory!
# For 1GB file → 1GB in memory!

# Method 2: .readline() - One line at a time
print("\n2. Using .readline() (line by line):")
with open("data.txt", "r") as file:
    print("Reading first 2 lines:")
    line1 = file.readline()
    line2 = file.readline()
    print(f"Line 1: {line1.strip()}")
    print(f"Line 2: {line2.strip()}")
# Memory efficient: only one line in memory at a time
```

```
# Method 3: .readlines() - All lines as list
print("\n3. Using .readlines() (list of lines):")
with open("data.txt", "r") as file:
    lines = file.readlines()
    print(f"Type: {type(lines)}") # Output: <class 'list'>
    print(f"Number of lines: {len(lines)}")
    print("First line:", lines[0].strip())
# Reads entire file, but as list of lines

# Method 4: Iterating directly over file object (BEST PRACTICE)
print("\n4. Iterating directly (recommended):")
line_count = 0
with open("data.txt", "r") as file:
    for line in file: # Most memory efficient
        line_count += 1
        print(f"Line {line_count}: {line.strip()}")
        if line_count >= 3: # Stop after 3 lines for demo
            break
print(f"Total lines processed: {line_count}")
```

Example - Context Managers (The Golden Rule)

```
# BAD: Manual file handling (risky!)
print("\nBAD PRACTICE (manual handling):")
try:
    file = open("test_bad.txt", "w")
    file.write("Some data\n")
    # What if an error occurs here?
    # File might not close properly!
    raise Exception("Oops! Something went wrong!")
    file.write("More data\n")
except Exception as e:
    print(f"Error occurred: {e}")
finally:
    # We might forget this!
    file.close()
    print("Manually closed file (if we remember)")
```

```
# GOOD: Using with statement (automatic cleanup)
print("\nGOOD PRACTICE (with statement):")
try:
    with open("test_good.txt", "w") as file:
        file.write("Some data\n")
        # Even if error occurs, file will close
        raise Exception("Oops! Something went wrong!")
        file.write("More data\n")
except Exception as e:
    print(f"Error occurred: {e}")
    print("File was automatically closed by 'with' statement!")
```

The **finally block** in Python **always executes**, regardless of whether an exception **occurred or not, caught or not**. It's used for cleanup operations that must happen no matter what.

The **raise keyword** in Python is used to **manually trigger or "throw" an exception**. When you raise an exception, you're signaling that something unexpected or erroneous has occurred in your program.